

called *NetAnim*, a standalone application, and it uses the custom trace files (for animation) generated by ns-3 to graphically display the simulation scenario. *NetAnim* is based on the *Qt4 GUI* toolkit. The animation trace file format generated by ns-3 for *NetAnim* will be discussed in detail in subsequent issues.

Another major difference between ns-2 and ns-3 is the way they have reached their present states, over a period of time. The difference is that ns-2 is software that has evolved, whereas ns-3 is planned software. Let me explain. REAL simulator was initially a single person's venture and later on, more code was added to it to create ns-1. Then ns-1 was remodelled and modified to ns-2. In the beginning, nobody had any idea about the eventual fate of ns-2. Source code was added, bugs were fixed, patches were released and, eventually, the present day ns-2 evolved. But with ns-3, this process was altogether different. ns-3 was developed with clear goals and deadlines in mind. Consider the fact that any planned software worth its salt ought to have a mascot or an emblem, but you can't find one for ns-2. Figure 2 shows the emblem for ns-3. So the moral of the story is, 'Eventually planned software is going to be far more useful than software evolved over time.' Yes, you have to trust me on this one—ns-3 is much better than ns-2. Moreover, nowadays ns-2 is only lightly maintained whereas ns-3 is maintained very fervently. To confirm this fact, please check the mailing list of network simulator (ns); topics related to ns-2 are mostly ignored

whereas ns-3 topics often lead to heated debates.

Prerequisites for learning ns-3

Skills in certain programming languages and tools might be beneficial if you are planning to work with ns-3. A middle level understanding of C++ is the only essential requirement. A basic knowledge of socket programming with C++ is necessary for understanding ns-3, which I will cover in this series. Python based scripting is also supported in ns-3, so knowing the basics of Python will be a bonus. But remember, Python scripting is not essential in ns-3, so the lack of Python skills is not a barrier to learning ns-3. Two other tools used in conjunction with ns-3 are *Doxygen* and *Waf*. The former is a documentation generator tool. Knowing the basics of this tool will help you decode the ns-3 source files more easily. *Waf* is the build automation tool used by ns-3. A basic understanding of *Waf* is required for compiling ns-3 modules. I will discuss both these tools in the next issue. So, essentially, you can start this journey through ns-3 with me if you have middle level C++ skills, and a lot of patience. **END** 

By: Deepu Benson

The author is assistant professor in Amal Jyothi College of Engineering, Kanjirappally, Kerala. He maintains a technical blog at www.computingforbeginners.blogspot.in, and can be reached at deepumb@hotmail.com.

FEEDBACK FEEDBACK FEEDBACK FEEDBACK FEEDBACK FEEDBACK FEEDBACK FEEDBACK FEEDBACK FEEDBACK FEEDBACK FEEDBACK FEEDBACK FEEDBACK FEEDBACK FEEDBACK

We Value Your Feedback

We love to hear from you as **OSFY** consistently strives to make its content **informative, interesting and engaging.**



Please share your feedback/ thoughts/ views via email at osfyedit@efy.in

We welcome your comments/ suggestions and encourage you to send them to:
The Editor, D-87/1, Okhla Industrial Area, Phase 1, New Delhi-110020.

OpenSource
THE COMPLETE SOURCE FOR OPEN SOURCE ForYou

FEEDBACK FEEDBACK FEEDBACK FEEDBACK FEEDBACK FEEDBACK FEEDBACK FEEDBACK FEEDBACK FEEDBACK FEEDBACK FEEDBACK FEEDBACK FEEDBACK FEEDBACK FEEDBACK